B+ Tree Indexing of 10 Million Records: Memory-only, Disk-only and Memory+Disk

Name: Tushar Kant · **Roll No:** B23ES1033 · Oct 10, 2026

1. Introduction and objective

This project implements a B+ Tree index from scratch in C++17, without any index or database library, and compares three storage strategies on a dataset of 10,000,000 randomly shuffled records. All three strategies share the same tree code, so differences in the results come only from where the pages are stored.

The assignment asks for three implementations:

- **Memory-only B+ Tree:** every node lives in RAM.
- **Disk-only B+ Tree:** every node lives in a file and every page access goes to the file, with no cache.
- **Memory + Disk B+ Tree:** nodes live in a file, with a fixed-size LRU buffer pool in RAM.

For each implementation the report analyses insertion time, search time, memory usage, disk usage, tree height and disk I/O, and states the assumptions on page size, tree order and storage structure.

Headline result (10M records): the index is 1005.7 MB with height 4. Memory-only inserts at 1.07 µs per record and searches at 1.27 µs, but needs about 1 GB of RAM. Disk-only needs about 15 MB of RAM and takes about 25 µs per cold search. The buffered tree cuts page reads per insert from 3.69 to between 0.94 and 0.39, depending on pool size.

2. Dataset and assumptions

The index key is a unique alphanumeric string and the payload is random data. All design parameters below are fixed assumptions of the implementation and are chosen so that one tree node equals one disk page.

Parameter	Value	Reason
Page / node size	4096 bytes	Equals the NTFS default cluster size and the x86/x64 memory page size, so one node is one aligned I/O
Key	8 bytes, zero-padded, compared with memcmp	Alphanumeric keys of 8 characters give 62^8 possible values, enough for 10M unique keys
Payload	64 bytes, zero-padded	Fixed-width records keep node layout simple
Node header	8 bytes (is_leaf, num_keys, next_leaf)	Shared by leaf and internal nodes
Internal node capacity	340 keys, 341 children	(4096 − 8 − 4) / (8 + 4) = 340
B+ Tree order (fan-out)	341	Maximum children of an internal node
Leaf node capacity	56 records of 72 bytes	(4096 − 8) / 72 = 56
Leaf links	next_leaf page number	Leaves form a left-to-right chain for range scans
Metadata	Page 0	Root page, height, record count, leaf and internal counts, key/payload/page sizes
Data pages	Page 1 onwards	Allocated sequentially; byte offset = page number × 4096
Duplicate keys	Existing key is overwritten	The dataset keys are unique, so this never triggers on the real data
Deletes	Not implemented	Not required by the assignment

Dataset used for the reported numbers. The assignment CSV was not available when the benchmarks were produced, so a generator (tools/gen_dataset.cpp) created 10,000,000 unique random 8-character alphanumeric keys with 64-character random payloads in shuffled order, in the same key,payload CSV shape as the assignment. The CSV is 740 MB.

The key and payload widths are compile-time options (KEY_LEN, PAYLOAD_LEN, PAGE_SIZE), and the program refuses to run, rather than truncate silently, if the real file has longer keys or payloads.

3. B+ Tree design

The tree is page-based: nodes refer to each other by page number, never by pointer, which lets the same code run on RAM or on a file. It talks to storage only through three calls: get(page) returns a page, mark_dirty(page) records a modification, and alloc() returns a new zeroed page.

Node layout. Every page is exactly 4096 bytes. An internal node stores 341 child page numbers followed by 340 separator keys. A leaf stores 56 keys followed by 56 payloads, plus the page number of its right sibling. Keys are compared as raw bytes, which gives lexicographic order.

Search. Starting at the root, each internal node is binary-searched for the child whose key range contains the search key. After height − 1 steps the leaf is binary-searched for an exact match. A search therefore reads exactly one page per level, which is 4 pages for 10M records.

Insert. Insertion is a recursive top-down descent to the correct leaf.

1. If the leaf has room, the record is inserted in sorted position.
2. If the leaf is full, its 56 records plus the new one (57) are split into 28 and 29 records, and the first key of the right leaf is copied up to the parent.
3. If the parent is also full, it is split the same way, but the middle key moves up instead of being copied. With 341 keys after the insert, the halves hold 170 and 170 keys.
4. If the root splits, a new root with one key and two children is created and the height grows by one.
5. If the key already exists, its payload is overwritten and the record count is unchanged.

Leaf chain. Splitting a leaf links the new page into the chain, so a range query descends once and then follows next_leaf pointers. A range_count function using this chain is included and tested.

Frame-safety rule. A page pointer returned by get() is never used after another get() or alloc(), because a store may recycle that memory. The code keeps page numbers across calls and re-fetches pages, which removes a class of stale-pointer bugs in the disk-backed stores.

Expected size. Random inserts into a B+ Tree leave leaves about 69% full on average (ln 2). For 10M records this predicts about 10,000,000 / (56 × 0.69) ≈ 259,000 leaves, and the measured value is 256,419 leaves (69.6% fill) plus 1,029 internal nodes.

4. Three storage back-ends

The tree class is a C++ template over a Store type, so the three implementations differ only in the store. Each store reports its own I/O counters.

Property	Memory-only (MemStore)	Disk-only (DiskStore)	Memory + Disk (BufferedStore)
Where pages live	RAM arena in 64 MB chunks	Index file only	Index file plus RAM buffer pool
Page cache	Not applicable, all pages resident	None: 2 scratch pages	LRU pool of N pages (default 4096 = 16 MB)
Page read	Pointer arithmetic	One pread per page not in scratch	Pool hit, or one pread on a miss
Page write	None	Write-through: one pwrite per modified page	Write-back: only on eviction or final flush
Lookup structure	Chunk index	Two-slot scan	Open-addressing hash table, page number to frame
New page	Zeroed memory	Zeroed scratch page, marked dirty	Zeroed frame in the pool, marked dirty
Persistent	No	Yes	Yes

Memory-only. Pages are allocated in 64 MB chunks that never move, so pointers stay valid and no large copy happens as the tree grows.

Disk-only. The two scratch slots exist only so the code can look at one page and the page it just touched. Every other access is a real file read, and every modification is written immediately. This models a tree with no buffer manager.

Memory + Disk. Frames form an intrusive doubly linked LRU list. A hash table maps page numbers to frames. When the pool is full, the least recently used frame is evicted, and written to the file only if it was modified. Pool sizes of 16 MB, 64 MB and 256 MB (4096,

16384 and 65536 pages) were benchmarked.

File access. The same wrapper issues pread/pwrite on Linux and ReadFile/WriteFile with an offset on Windows, checks every return value and stops on an I/O error. A cold option flushes the pool and asks the Linux kernel to drop its cached copy of the index file before searching, so search latency reflects the device and not the OS cache.

5. Development process and decisions

The work followed five steps: a first implementation, a code review, a rewrite that fixed the review findings, a test suite, and full-scale benchmarking.

Key design decisions

- **One tree, three stores.** A template parameter keeps the algorithm identical across modes, so the comparison is fair and a bug is fixed once.
- **Page numbers instead of pointers.** This is what makes the same nodes storable in RAM or on disk.
- **Fixed-width keys and payloads.** They give a constant leaf and internal capacity, and make node arithmetic exact.
- **Header kept in RAM, written to page 0.** The 40-byte header is cached in the tree object and written when the root changes and once at the end, instead of rewriting page 0 on every insert, which would have inflated disk-only write counts.
- **No double writes for new pages.** A new page is created zeroed in memory and marked dirty, and reaches the file once, when it is written or evicted.
- **Write-through for disk-only, write-back for buffered.** This is the defining difference between the two disk-backed modes.
- **Fail loudly, never truncate.** Keys or payloads longer than the compiled widths abort the run with the exact rebuild command.

Problems found in the first version and how they were fixed

Problem	Effect	Fix
Keys and payloads silently truncated to 8 and 64 bytes	Longer keys collide, so records are lost and sizes are understated	Reader aborts on over-long keys and payloads; widths configurable at build time
Record count increased on duplicate overwrite	Count could disagree with the leaf chain	insert() returns whether a new key was added

Problem	Effect	Fix
Stale metadata pointer after a root split in disk mode	About a 1-in-16 chance of corrupting the new root on each root split	Header held in the tree object; the old root is saved before any page fetch
Dead, out-of-bounds code in the internal split	Read past the end of the key array	Removed; split rewritten with memmove and a temp area
CSV reader mishandled CRLF at buffer boundaries and stopped on blank lines	Corrupt record or early stop	Line-based reader handling LF, CRLF, blank lines and an optional header
Missing-key test altered the last character	The altered key might exist, giving false errors	Absent keys contain a control byte that cannot be in the data
Node counts skipped for large disk runs	Leaf and internal counts reported as 0	Counters maintained during insert
Page 0 rewritten every insert, new pages written twice	Inflated write counts	Header cached; new pages written once
Insert time included CSV parsing	Timing mixed I/O of the input file with index work	Per-insert timing; parsing time reported separately
Search check did not compare payloads	Wrong payloads could pass	Sample payloads stored and compared

6. Experimental methodology and metrics

Test machine. Linux sandbox with 1 vCPU, 3 GB RAM, a virtio-backed ext4 disk, compiled with g++ 13 at -O2. Absolute times will differ on other machines, but the I/O counts and the ratios between modes are properties of the algorithm.

Procedure for each run

1. Stream the CSV once and insert records in file order (the order is random). Each insert is timed individually.
2. Flush all dirty pages and fsync the index file. The flush time is added to the insertion time.

3. Verify the whole tree structure (see section 7).
4. If cold search is requested, empty the buffer pool and drop the OS file cache.
5. Reset I/O counters, then run 100,000 searches for keys that exist (every 100th inserted key) and 10,000 searches for keys that cannot exist.
6. Append one row of results to a CSV file.

Metrics

Metric	How it is measured
Insertion time	Sum of per-insert timings plus final flush; CSV parsing excluded. Wall time including parsing is also reported
Search time	Average, median (p50) and 99th percentile over 110,000 queries
Memory usage	Peak resident set size of the process (VmHWM on Linux, PeakWorkingSetSize on Windows)
Disk usage	Number of allocated pages × 4096 bytes
Tree height	Levels from root to leaf, root-only tree = 1
Node counts and leaf fill	Counters maintained by the tree; fill = records / (leaves × 56)
Disk I/O	Pages and MB read and written, counted at the file-access layer, separately for the insert and search phases
Buffer behaviour	Pool hits, misses and hit ratio, separately for the insert and search phases

Throughput. Throughput is computed as one divided by the average time per operation.

Limitation of the timing data. The page counts are exact. Elapsed times include the operating system's file cache for the insert phase, which is discussed in section 9.

7. Correctness verification

The tree indexed all 10,000,000 records correctly in every mode. Correctness was checked at three levels.

1. Self-test suite (make test). Each case inserts keys into the tree and into a std::map reference (used only inside the test), then compares them.

- Random, ascending and descending key orders, on all three stores, with up to 200,000 keys.
- A buffered store with only 4 pages, which forces constant eviction and reload.
- Overwriting 10% of existing keys: the insert must report not-new, and the latest payload must be returned.
- Record count equals the number of distinct keys.
- Every key is found with its latest payload, and 2,000 absent keys are not found.
- 200 random range counts equal the reference.
- After dropping all caches, every key is read back correctly from disk.
- Boundary sizes: 1, 2, 56 (full leaf), 57 (first split) and 168 records.

2. Stress build. The same suite is compiled with 256-byte pages (3 records per leaf, 20 keys per internal node) under AddressSanitizer and UndefinedBehaviorSanitizer. Tiny pages make the tree 5 levels deep with only 60,000 keys, so internal splits and root growth happen thousands of times. All tests passed with no memory or undefined-behaviour reports.

3. Checks inside every benchmark run. After inserting, the program walks the entire tree and verifies:

- keys are strictly increasing inside every node;
- every key lies within the range given by its parent's separator keys;
- all leaves are at the same depth;
- the leaf chain is strictly sorted and contains exactly the recorded number of records and leaves.

It then searches 100,000 existing keys, comparing the returned payload with the inserted one, and 10,000 keys that cannot exist.

Outcome for all five 10M configurations: structure check OK with 10,000,000 records in the leaf chain, 100,000 of 100,000 existing keys found with matching payloads, 0 payload mismatches, and 0 of 10,000 false positives.

The structure check proves all 10M records are present, ordered and reachable. The 10M benchmark does not look up every one of the 10M keys individually.

8. Results

All results are for 10,000,000 records (synthetic, 8-byte keys and 64-byte payloads) on the machine in section 6. Disk-backed searches were run cold. Raw numbers are in results_synthetic_10m.csv in the repository. MB means 1,048,576 bytes.

8.1 Time and throughput

Configuration	Insert total (s)	Insert avg (µs)	Insert throughput (records/s)	Search avg (µs)	Search p50 (µs)	Search p99 (µs)	Search throughput (queries/s)
Memory-only	10.69	1.07	936,000	1.27	1.14	1.82	787,000
Disk-only	38.22	3.82	261,700	24.94	27.50	61.66	40,100
Memory + Disk, 16 MB pool	38.09	3.81	262,500	23.64	26.08	58.84	42,300
Memory + Disk, 64 MB pool	38.44	3.84	260,200	24.34	27.63	69.13	41,100
Memory + Disk, 256 MB pool	28.18	2.82	354,900	24.07	26.27	68.86	41,500

Insert times exclude CSV parsing. Including parsing, wall times were 11.47 s, 39.13 s, 39.05 s, 39.42 s and 29.15 s respectively.

8.2 Tree structure, memory and disk usage

Configuration	Height	Leaf nodes	Internal nodes	Leaf fill	Index size (MB)	Peak process memory (MB)	Memory held by the store (MB)	Disk usage (MB)
Memory-only	4	256,419	1,029	69.6%	1005.7	1020.7	1005.7	0
Disk-only	4	256,419	1,029	69.6%	1005.7	15.1	0	1005.7
Memory + Disk, 16 MB pool	4	256,419	1,029	69.6%	1005.7	31.2	16.1	1005.7
Memory + Disk, 64 MB pool	4	256,419	1,029	69.6%	1005.7	79.6	64.5	1005.7
Memory + Disk, 256 MB pool	4	256,419	1,029	69.6%	1005.7	273.1	258.0	1005.7

The tree has 257,449 pages in total (1,054,511,104 bytes) including the metadata page. The raw data is 720 MB (10M × 72 bytes), so the index is about 1.4 times the raw data.

8.3 Disk I/O during insertion

Configuration	Page reads	Page writes	MB read	MB written	Reads per insert	Writes per insert	Pool hit ratio
Memory-only	0	0	0	0	0	0	not applicable
Disk-only	36,868,827	10,514,892	144,019	41,074	3.687	1.051	not applicable
Memory + Disk, 16 MB pool	9,359,897	9,516,006	36,562	37,172	0.936	0.952	74.97%
Memory + Disk, 64 MB pool	7,590,147	7,847,596	29,649	30,655	0.759	0.785	79.70%
Memory + Disk, 256 MB pool	3,903,515	4,160,964	15,248	16,254	0.390	0.416	89.56%

8.4 Disk I/O during search (110,000 queries, cold)

Configuration	Page reads	Reads per query	Pool hits	Pool misses	Pool hit ratio
Memory-only	0	0	not applicable	not applicable	not applicable
Disk-only	440,000	4.000	not applicable	not applicable	not applicable
Memory + Disk, 16 MB pool	114,699	1.043	325,301	114,699	73.93%
Memory + Disk, 64 MB pool	104,797	0.953	335,203	104,797	76.18%
Memory + Disk, 256 MB pool	92,005	0.836	347,995	92,005	79.09%

8.5 Scaling from 1M to 10M records

Earlier runs with 1,000,000 records on the same data shape show how cost grows with N. The 16 MB buffered row used a cold search.

Configuration	Insert 1M (s)	Insert 10M (s)	Search avg 1M (µs)	Search avg 10M (µs)
Memory-only	0.64	10.69	0.75	1.27
Disk-only (cold search)	2.19	38.22	14.65	24.94
Memory + Disk, 16 MB pool	1.67	38.09	12.49	23.64

At 1M records the tree has height 3 (25,597 leaves, 129 internal nodes, 100.5 MB). At 10M it has height 4 and is 10 times larger.

9. Analysis and discussion

Tree height and size. Ten million records need a tree of height 4 because an internal node fans out to about 250 children on average (256,419 leaves under roughly 1,024 bottom-level internal nodes), not the maximum 341. The 4 levels above the records cost almost nothing in space: the 1,029 internal nodes occupy about 4 MB, 0.4% of the 1005.7 MB index. The index is about 1.4 times the 720 MB of raw data because leaves are on average 69.6% full, close to the theoretical 69% for random insertion, and because each page carries a header.

Memory-only versus disk-backed. Memory-only is the fastest: 1.07 µs per insert and 1.27 µs per search, about 936,000 inserts per second. The price is memory: its peak was 1020.7 MB, about 68 times the 15.1 MB used by disk-only. Disk-only inserted 3.6 times slower and searched about 20 times slower (24.94 µs against 1.27 µs), but it uses almost no RAM and its index survives the process.

Disk-only I/O. Without a cache, a search touches exactly one page per level: 440,000 reads for 110,000 queries, which is 4.000 reads per query and equals the tree height. An insert needed 3.69 reads and 1.05 writes on average. It reads fewer than 4 pages because the two scratch pages sometimes already hold the next page, and it writes about one page per insert (the modified leaf) plus extra writes when a split also updates the new sibling and the parent.

Effect of the buffer pool. Even a small pool helps a lot, because the internal levels are tiny and are used by every operation.

- A 16 MB pool (1.6% of the index) cut insert page reads from 3.69 to 0.94 per insert, a 75% reduction, and search reads from 4.0 to 1.04 per query.

- A 64 MB pool reached 0.76 reads per insert (79% fewer) and a hit ratio of 79.7%.
- A 256 MB pool reached 0.39 reads per insert (89% fewer) and a hit ratio of 89.6%.

Returns diminish because insertion keys are random: each insert lands in an unpredictable one of 256,419 leaves, so the pool keeps the hot internal pages but can hold only a fraction of the leaves. Writes fall more slowly than reads (0.95, 0.79 and 0.42 per insert), because every leaf that changed must eventually be written once when it is evicted, even if the later reads are avoided.

Search latency of the disk-backed modes. Cold disk-only and buffered searches have similar average latency (about 24 µs). This is consistent with each query needing one real device read for its leaf, which is random among 256,419 pages, in both modes. The upper levels are cheap in both modes, held either in the buffer pool or in the operating system's cache after the first few queries, so the only difference between the modes is whether those upper-level reads are counted by the store (4.0 reads per query in disk-only against about 1 in buffered).

Reading the insertion times for disk-only and buffered. Disk-only and the 16 MB and 64 MB pools show almost the same insertion time (about 38 s), even though buffered issues far fewer reads. In these runs the operating system's file cache absorbs most reads and writes issued by the program, so an I/O call costs a system call and a memory copy, not a device access. For that reason the I/O counts are the reliable comparison between the disk-backed modes. On a system without such caching, or with direct I/O, the larger read reduction would be expected to show up more strongly in elapsed time. This was not measured here. Only the 256 MB pool, with 0.39 reads per insert, was clearly faster (28.2 s).

Throughput summary. Memory-only sustains about 936,000 inserts per second and 787,000 searches per second. The disk-backed modes sustain about 260,000 to 355,000 inserts per second and about 40,000 to 42,000 cold searches per second.

Choosing a mode.

- Memory-only suits an index that fits comfortably in RAM and does not need to persist; at 10M records it needs about 1 GB.
- Disk-only suits very small memory budgets where latency does not matter, but it pays 4 page reads for every search.
- Memory + Disk is the practical choice: memory use is a tunable setting, the index is persistent, and a pool of a few percent of the index already removes about three quarters of the insert reads.

10. Limitations, conclusion and reproduction

10.1 Limitations

- **Synthetic data.** The reported numbers come from generated data with 8-byte keys and 64-byte payloads. Results on the assignment CSV may differ in absolute terms, and the widths may need adjusting with build options.
- **Single run per configuration.** Each configuration was run once, so no confidence intervals are given. Times can vary by a few percent between runs; for example two memory-only runs took 11.0 s and 10.7 s.
- **Operating system cache.** Insertion times for the disk-backed modes are influenced by the OS file cache, as discussed in section 9. Page counts are not affected.
- **Platform.** Benchmarks ran on Linux with one CPU. The Windows file-access path (ReadFile and WriteFile) is implemented but was not run in these experiments, and the cold-cache option only drops the OS cache on Linux.
- **Scope of the index.** Fixed-width keys and payloads, no deletion, no concurrency control and no crash recovery. These were outside the assignment.
- **Sampled lookups.** The benchmark searches 100,000 existing keys, not all 10 million. Presence and ordering of all records is covered by the full structural check.

10.2 Conclusion

A B+ Tree built from scratch with 4096-byte pages, 341-way internal nodes and 56-record leaves indexes 10,000,000 records in 4 levels and 1005.7 MB. It was verified correct in all three storage modes. Memory-only is fastest (1.07 µs insert, 1.27 µs search) but needs about 1 GB of RAM. Disk-only needs about 15 MB of RAM and reads one page per level on every access. The buffered tree gives a tunable middle ground: a pool holding 1.6% of the index removes 75% of insert reads, and a pool holding 25% removes 89%.

10.3 Reproduction

The source is a standard C++17 project (no third-party libraries). Requirements are g++ and make; on Windows, MinGW-w64 or MSYS2.

```
make                          # build ./bpt  (bpt.exe on Windows)
make test                     # self-tests, 4 KB pages and 256-byte stress
build
./bpt inspect data.csv        # check key and payload lengths against the
build
./run_all.sh data.csv         # tests plus all benchmark configurations ->
results.csv

# individual runs
./bpt mem  data.csv
./bpt disk data.csv --cold
./bpt buf  data.csv --cache-pages 4096 --cold
```

If the CSV has keys longer than 8 characters or payloads longer than 64 bytes, rebuild with, for example, make KEY_LEN=12 PAYLOAD_LEN=100.

10.4 Repository contents

Path	Purpose
src/bptree.hpp	B+ Tree node layouts, insert, search, range count and verification
src/stores.hpp	MemStore, DiskStore and BufferedStore, plus the portable file wrapper
src/main.cpp	Command line, CSV reader, benchmark, inspect mode and self-tests
tools/gen_dataset.cpp	Optional generator for test data
Makefile, run_all.sh, run_all.bat	Build, test and benchmark scripts
README.md	Build, usage and assumptions
results_synthetic_10m.csv	Raw measurements behind section 8